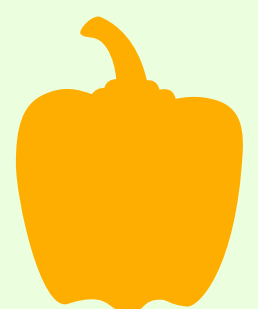
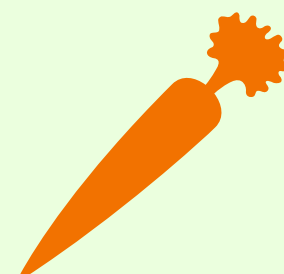
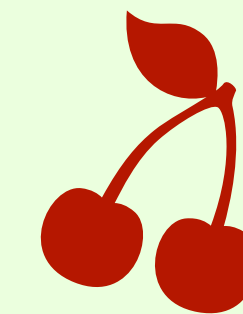
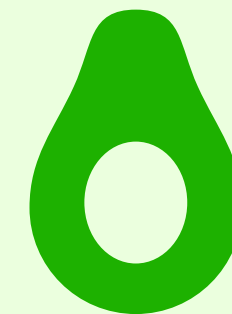
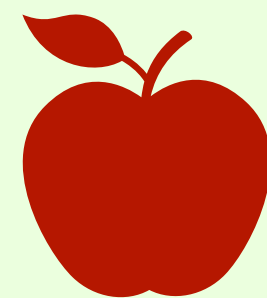
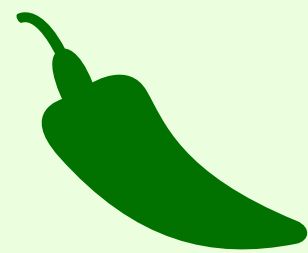


PerishABit

Nishka Pandya



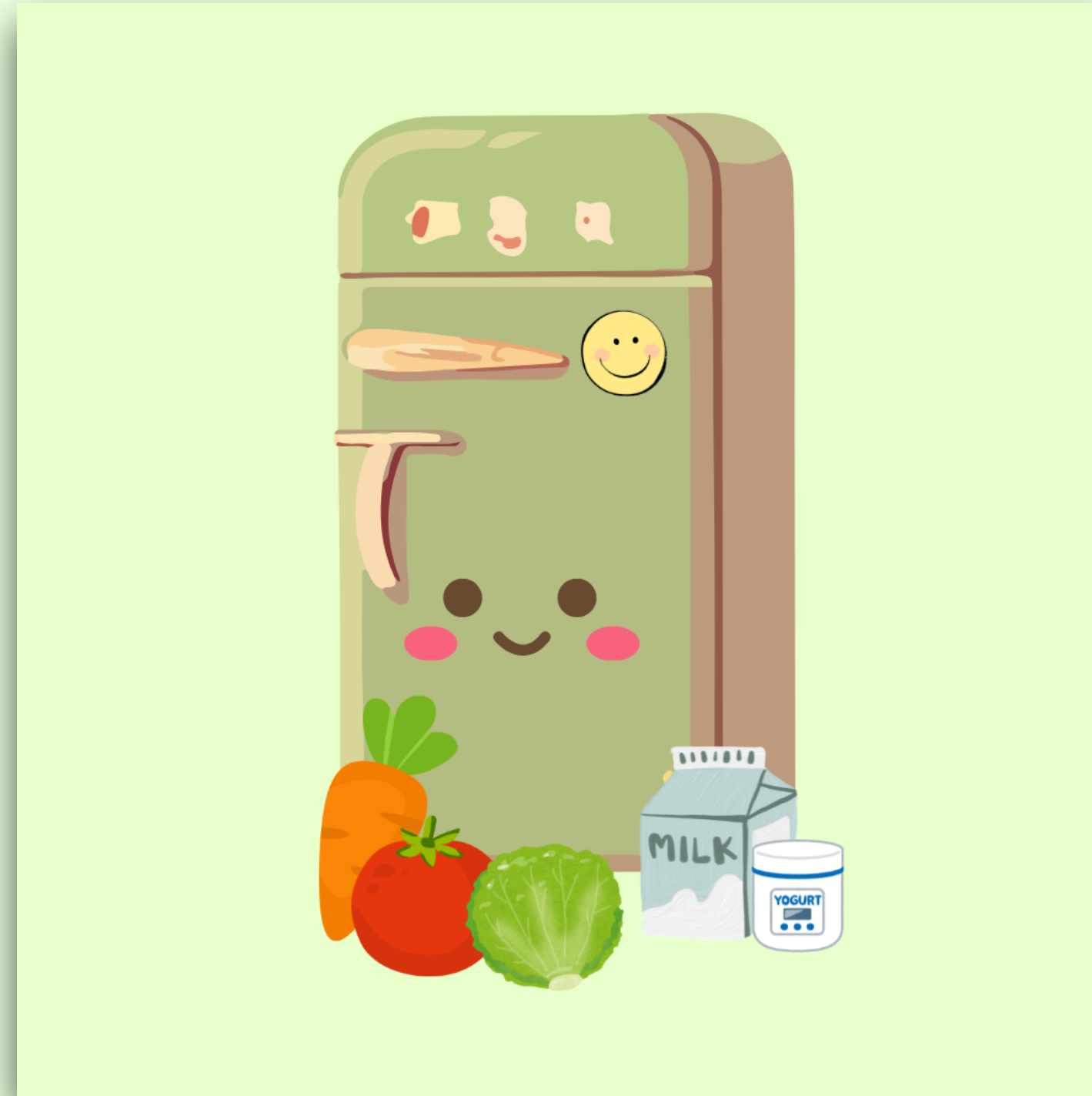
Imagine...



Looking in your refrigerator, impulsively eating items from the front of the fridge while disregarding items in the back



The items in the back get forgotten and expire or spoil, leading to repeated food waste and loss of money



PerishABit

Reduce perishable food waste a bit at a time

The Problem

- Food waste emits greenhouse gases in waste fields
- Two of the causes of food wastes among households are disorganization and forgetfulness
- Young adults tend to struggle the most with reducing food waste



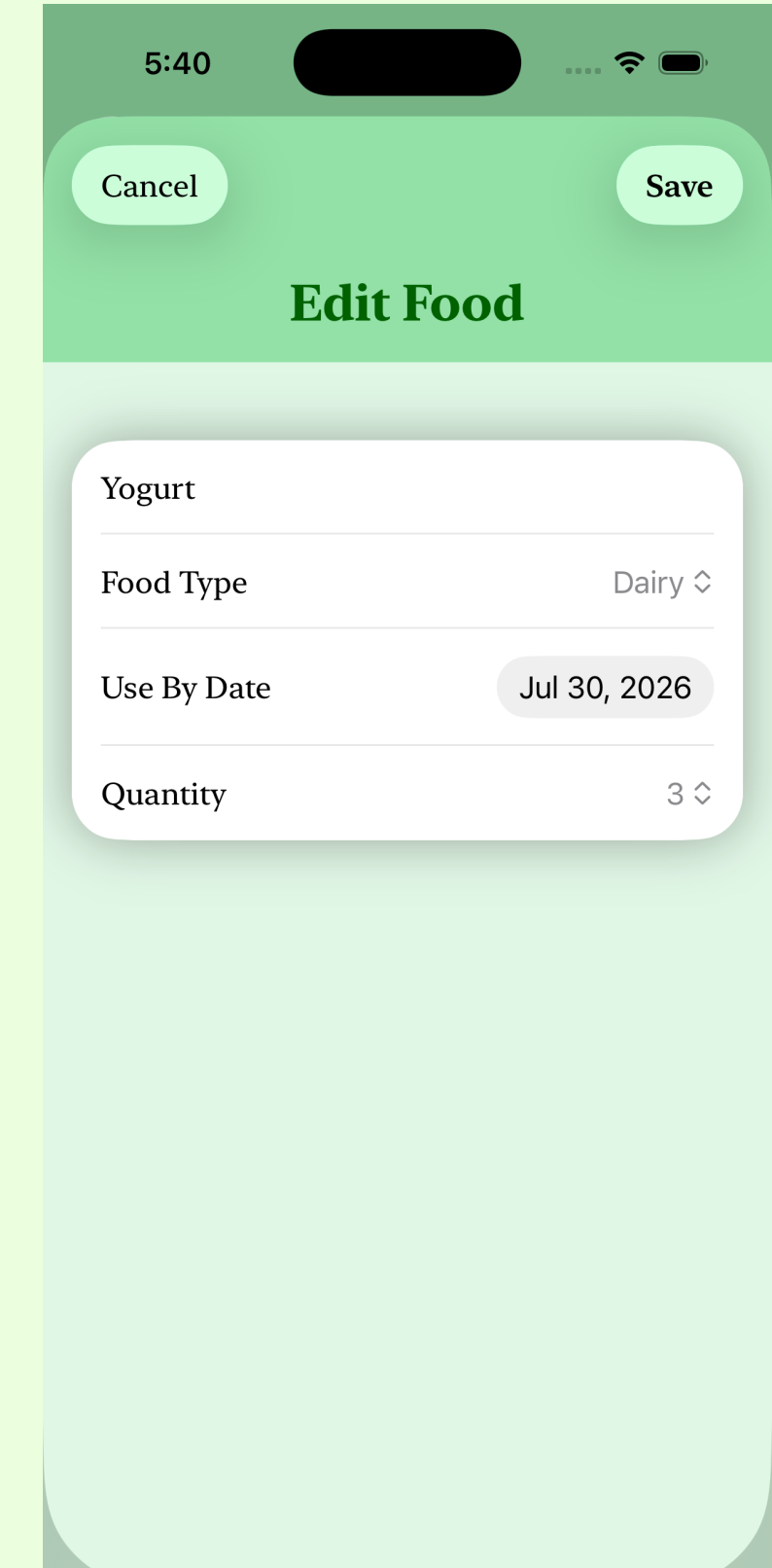
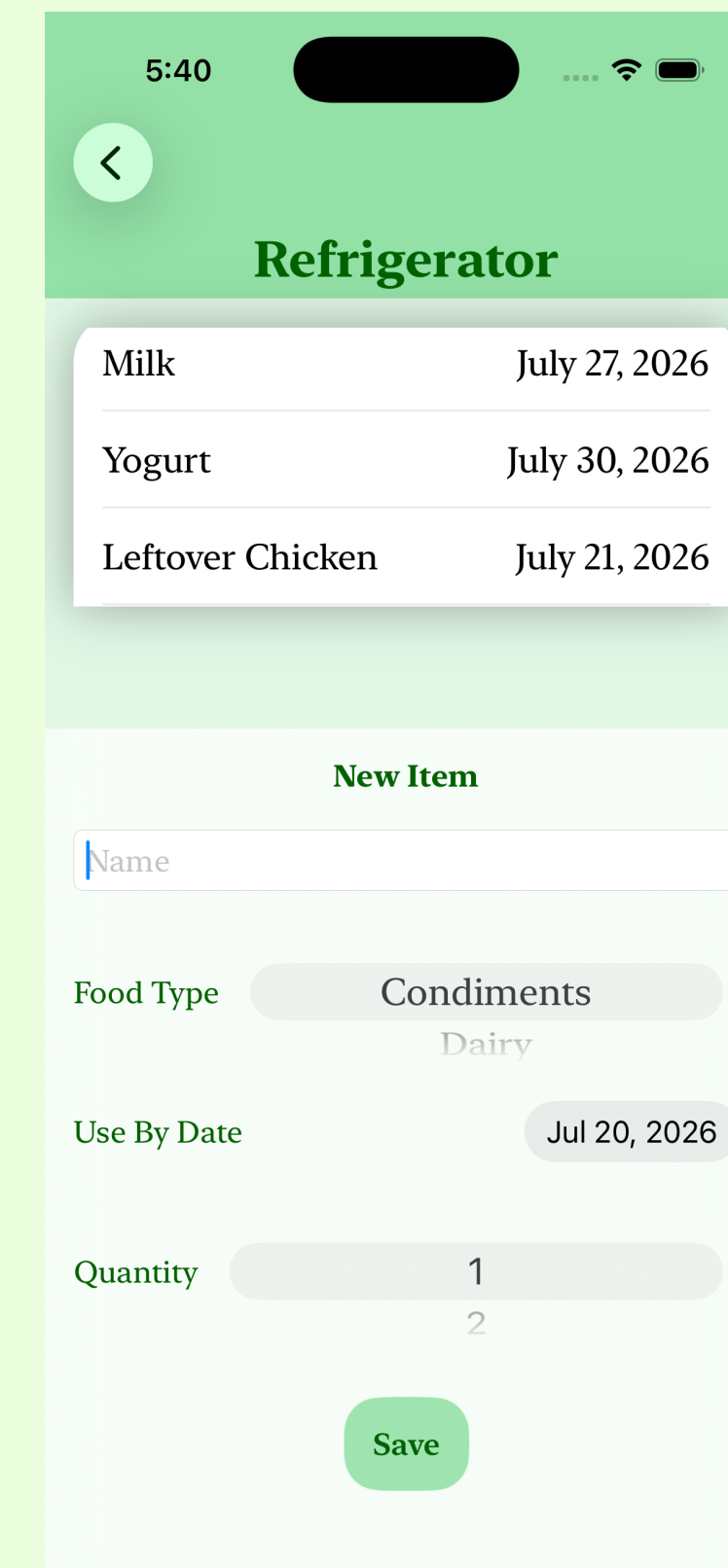
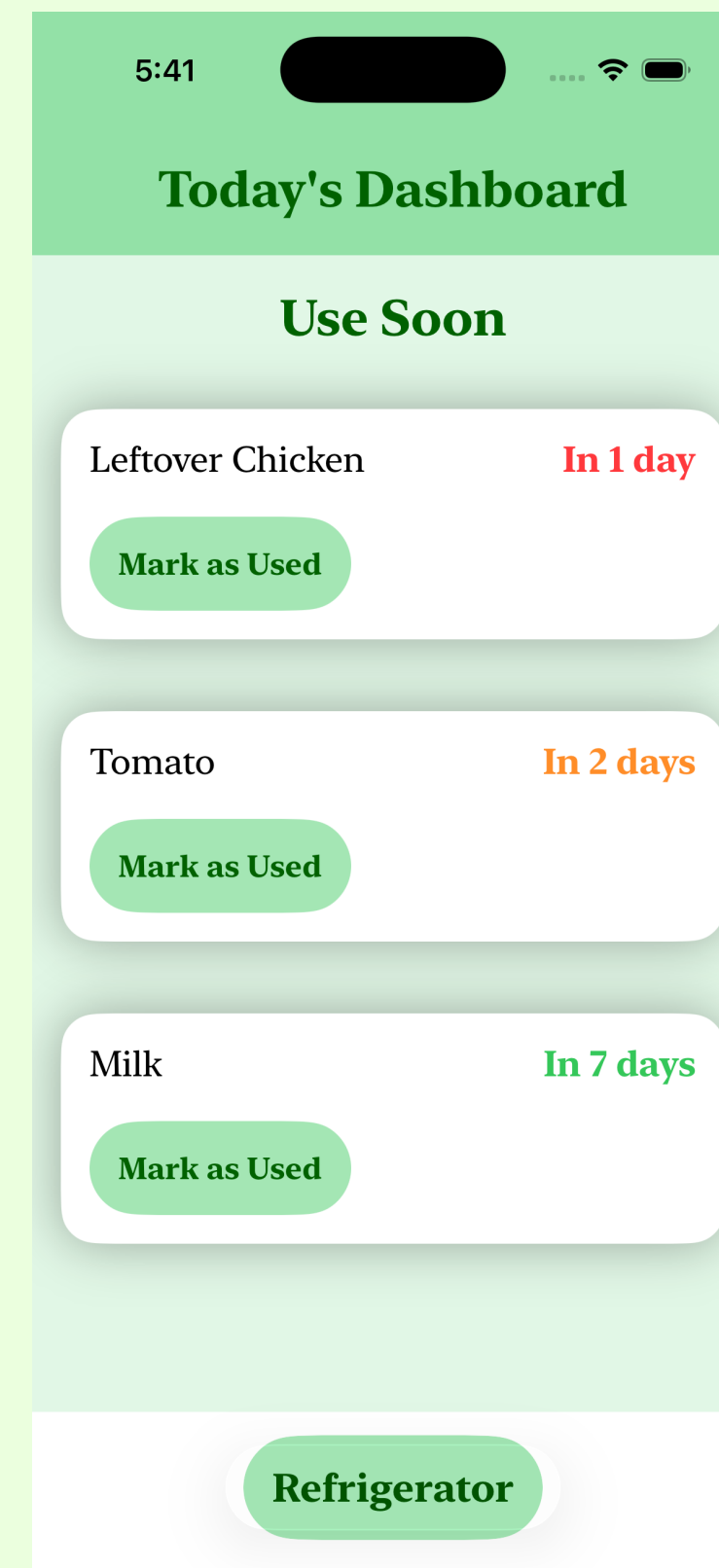
The Solution

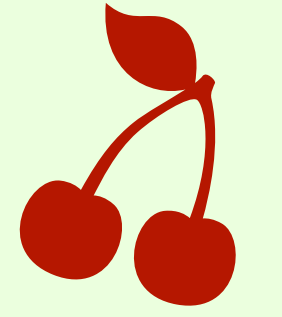
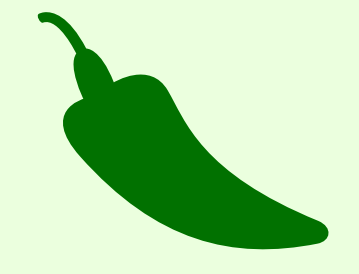
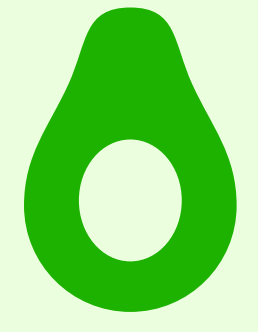
- Targets young adults, providing a space to track perishable food items in their refrigerator and prioritize the most important food items to use
- Designed to reduce food waste by encouraging young adults to develop organization habits



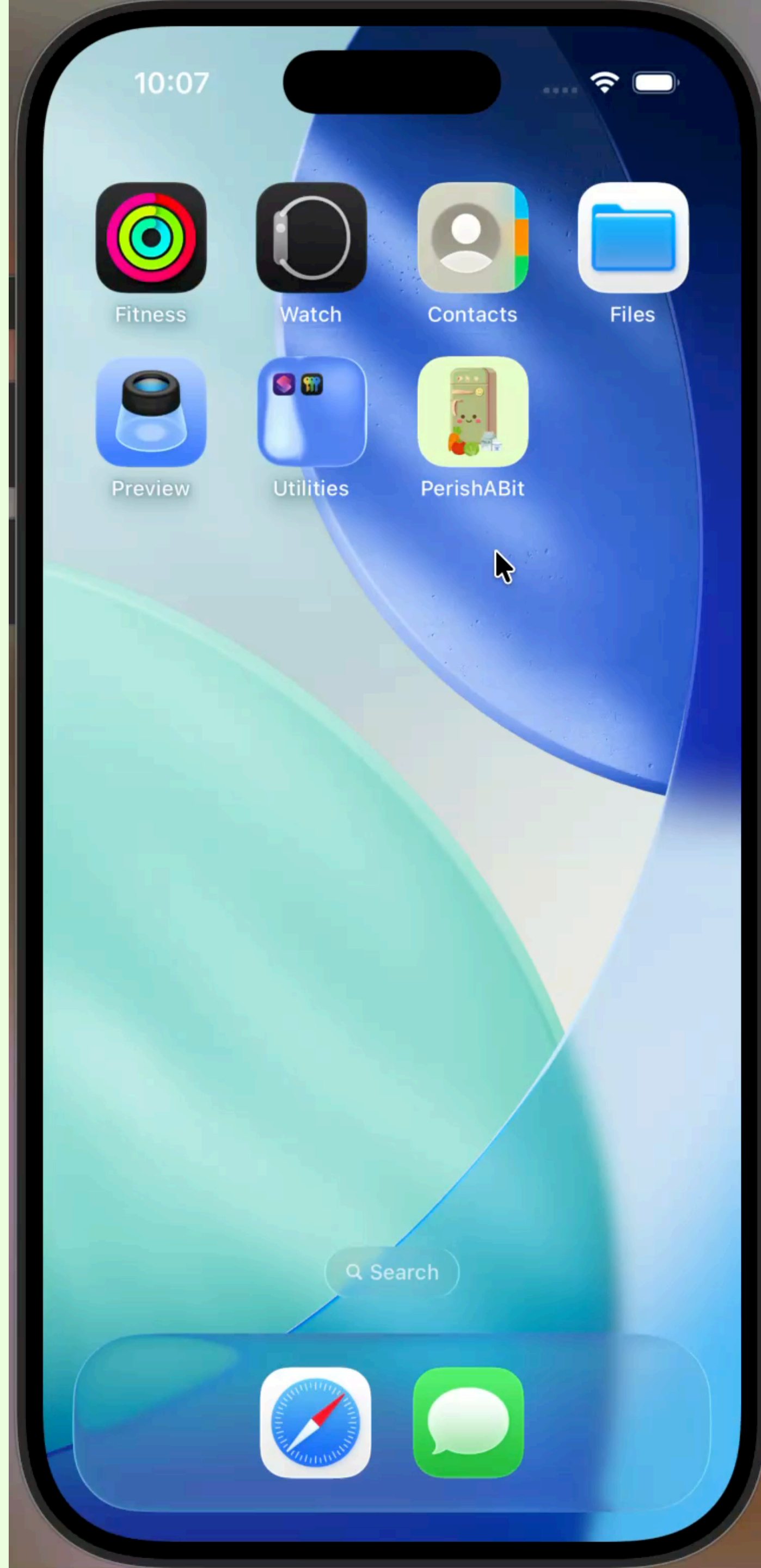
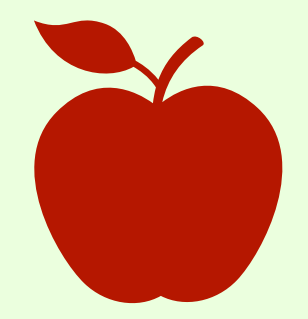
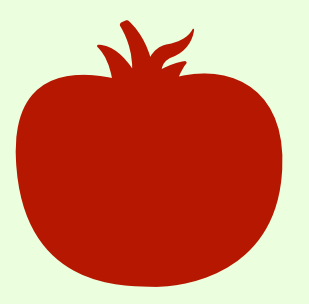
Features

- 3 screens - designed to be simple and not overwhelming
- Can add various types of perishable items as well as their desired use by date
- Top 3 items will be added to the dashboard based on items added
- Can delete and update food items





Live Demo



Code Spotlight

- **SwiftData**
- **Function:** Finding top 3 food items to prioritize
- **SwiftUI**
- **If-Else Statements:** Makes dashboard interactive

```
func findTop(originalFoodArray: [FoodItemInfo]) -> [FoodItemInfo]{
    var topFoodItems = [FoodItemInfo]()

    let sortedByDate = originalFoodArray.sorted(by: {$0.useByDate < $1.useByDate})

    var index = 0;
    while (topFoodItems.count < 3 && index < sortedByDate.count){
        topFoodItems.append(sortedByDate[index])
        //sortedByDate.remove(at: index)
        index+=1
    }

    return topFoodItems
}
```

```
if(findTop(originalFoodArray: foodDashboard).count <= 0){
    //Text("hi")
    VStack(alignment:.leading, spacing: 20.0){
        Text("Add New Groceries in Refrigerator!")
            .fontDesign(.serif)
            .frame(minWidth:120, maxWidth: .infinity, alignment:.center)
            .font(.title3)
    }
    .padding()
    .background(Rectangle().foregroundColor(.white))
    .cornerRadius(20)
    .shadow(radius: 15)
    .padding()
}

else{
    ForEach(findTop(originalFoodArray: foodDashboard)){ food in
        VStack(alignment:.leading, spacing: 20.0){
            HStack(spacing: 100.0){
                Text(food.name)
                    .fontDesign(.serif)
                    .frame(minWidth:120, maxWidth: .infinity, alignment:.leading)
                    .font(.title3)
                let calendar = Calendar.current
                let startMidnight = calendar.startOfDay(for: Date.now) // Used ChatGPT to learn
                //how to convert from date to dateComponents
                let endMidnight = calendar.startOfDay(for: food.useByDate)
                let daysRemaining = calendar.dateComponents([.day], from: startMidnight, to:
                endMidnight).day!
                if (daysRemaining <= 1){
                    Text("In \((daysRemaining) day")
                        .foregroundColor(Color.red)
                        .fontWeight(.bold)
                }
                else if (daysRemaining <= 3){
                    Text("In \((daysRemaining) days")
                        .foregroundColor(Color.orange)
                        .fontWeight(.bold)
                }
                else{
                    Text("In \((daysRemaining) days")
                        .foregroundColor(Color.brown)
                        .fontWeight(.bold)
                }
            }
        }
    }
}
```

Code Spotlight

- **SwiftData:** Allow food items data to persist
- **CRUD Operation, functions & Food Class:** Allows users to have control over accurately adding and updating items according to their refrigerator

```
import Foundation
import SwiftData

@Model
class FoodItemInfo{

    var name: String
    var useByDate: Date
    var quantity: Int
    var type: String

    init(name: String, useByDate: Date, quantity: Int, type: String, ){
        self.name = name
        self.useByDate = useByDate
        self.quantity = quantity
        self.type = type
    }

}
```

```
.sheet(item: $selectedFood){ food in
    NavigationStack {
        EditFoodView(food: food)
    }
}
```

```
List{
    ForEach(foods) { food in
        HStack{
            Text(food.name)
                .font(.title3)
                .fontDesign(.serif)
            Spacer()
            Text(food.useByDate, format: .dateTime.month(.wide).day().year())
                .font(.title3)
                .fontDesign(.serif)
        }

        .onTapGesture {
            selectedFood = food
        }
    }
    .onDelete(perform: deleteItem)
}
```

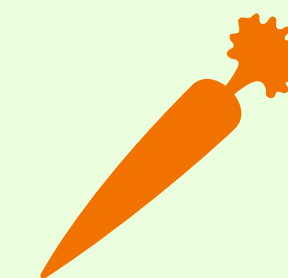
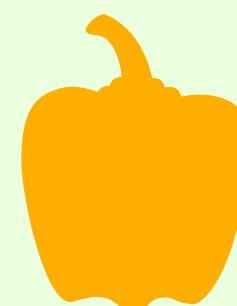
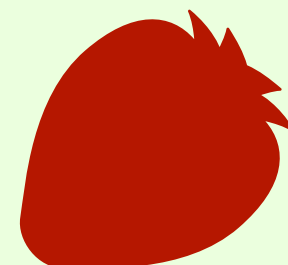
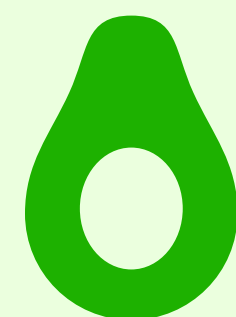
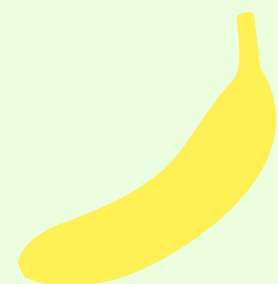
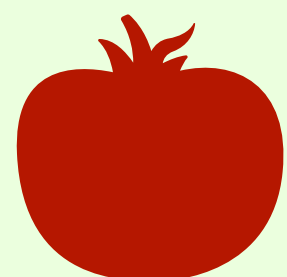
```
Button("Save"){
    let newFood = FoodItemInfo(name: newFoodItem, useByDate: newUseByDate, quantity:
        newQuantity, type: newType)
    context.insert(newFood)
    newFoodItem = ""
    newUseByDate = .now
    newQuantity = 1
    newType = ""
}
```

AI & External Use

- Prompted ChatGPT to provide tips with problem-solving and brainstorming debugging solutions or design choices without completely providing code or doing all the thinking
- ChatGPT helped better understand the roles of ModelContext, ModelContainer, and @Query
- Used Canva to design app icon

Future Enhancements

- Search feature
- Sorting by dates and categories
- Quick add that tracks the most used items
- Individual accounts
- Community feature allowing users to interact and share tips and find accountability partners to track food



Thank You

